

Memoria: Seminario 1

Trabajo realizado por:

Rafael Fernando Rivas Cabrera

Álvaro García Zabra

Diego Cabrera Martínez

Lenguaje utilizado

Para la realización de este seminario decidimos utilizar Python debido a que miembros del grupo ya se encontraban familiarizados con este lenguaje, lo que facilitó el transcurso del trabajo. Además, al tratarse de un lenguaje de programación bastante intuitivo y con una carga de aprendizaje menor, en comparación con otros, no supondría un problema para aquellos miembros que no lo hubiesen utilizado previamente.

Herramientas

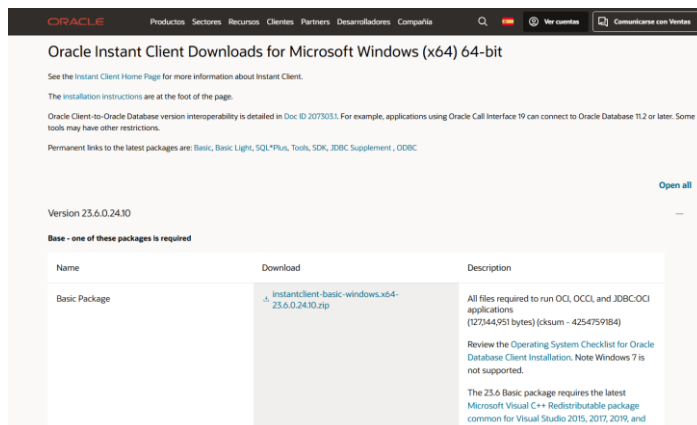
IDE: Debido a la familiarización de ciertos miembros con este, optamos por utilizarlo todo el equipo permitiendo una instalación rápida de tanto Python como los paquetes necesarios. Además, dispone de un apartado de base de datos desde el que podíamos visualizar los distintos elementos para ir comprobando los cambios desde el propio IDE sin necesidad de utilizar otras herramientas como SQLDeveloper.

Biblioteca para la utilización del driver ODBC: Decidimos utilizar pyodbc ya que era la biblioteca recomendada por el documento correspondiente a la presentación del seminario 1. Para su instalación simplemente tuvimos que descargarlo a través del IDE.

Biblioteca de la interfaz gráfica: Para este apartado utilizamos la biblioteca kivy la cual, a pesar de no ser la más sencilla de utilizar, ofrece una apariencia más moderna sin llegar a un nivel de complejidad muy elevado dando un toque más profesional a la interfaz. Para su instalación simplemente tuvimos que descargarlo a través del IDE.

Driver ODBC: Fue indispensable la instalación de un driver ODBC para poder ejecutar la comunicación con la base de datos, en nuestro caso, este se trató de ORACLE ODBC para cuya instalación tuvimos que seguir los siguientes pasos:

1. Creación de un directorio; C:\oracle (no es estrictamente necesario pero si recomendable)
2. Descargar el paquete Instant Client Basic desde la web oficial de oracle



3. Descargar el paquete Instant Client ODBC correspondiente al paquete instalado en el paso anterior (para obtenerlo simplemente bajamos en la página donde descargamos el paquete Instant Client Basic)

Development and Runtime - optional packages		
Name	Download	Description
SDK Package	instantclient-sdk-windows.x64-23.6.0.24.10.zip	Additional header files and an example makefile for developing Oracle applications with Instant Client (1,469,780 bytes) (cksum - 3761378787)
JDBC Supplement Package	instantclient-jdbc-windows.x64-23.6.0.24.10.zip	Additional support for Internationalization under JDBC (1,597,315 bytes) (cksum - 4081574687)
ODBC Package	instantclient-odbc-windows.x64-23.6.0.24.10.zip	Additional libraries for enabling ODBC applications (1,059,750 bytes) (cksum - 1338164586)
Precompiler Package	Precompiler Downloads	Additional files for Pro*C and Pro*COBOL

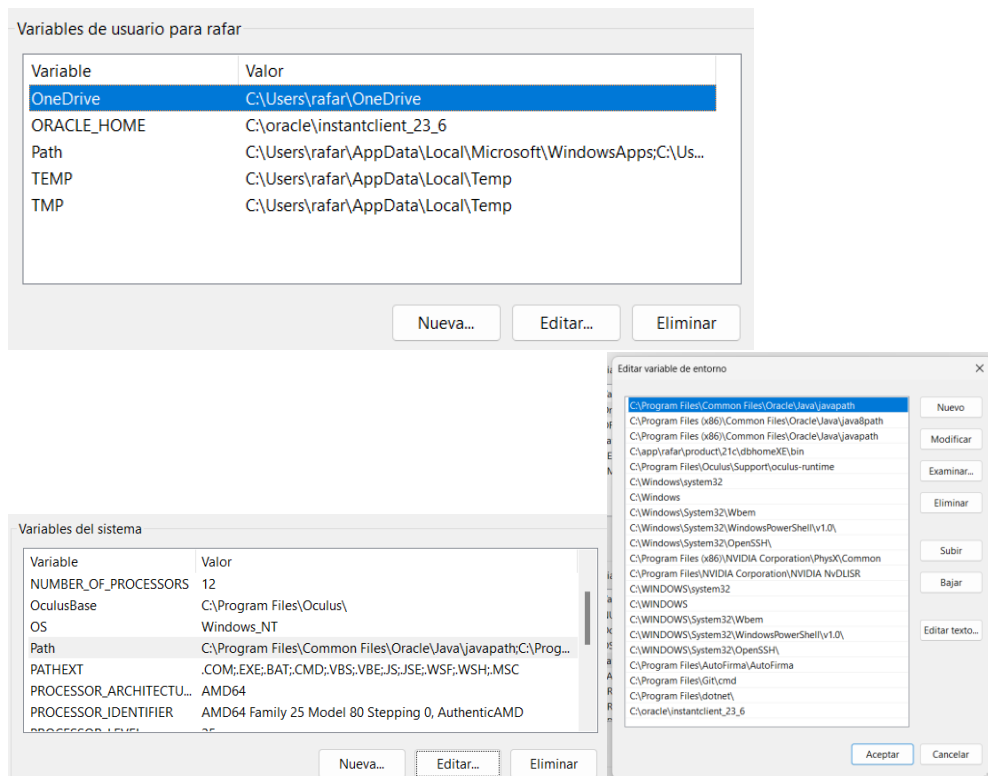
4. Descomprimos el archivo descargado en el paso 2 dentro directorio creado en el paso 1.

	instantclient-basic-windows.x64-23.6.0.24...	09/11/2024 13:42	Carpeta comprimi...	124,165 KB
	instantclient_23_6	09/11/2024 13:54	Carpeta de archivos	
	META-INF	09/11/2024 13:50	Carpeta de archivos	

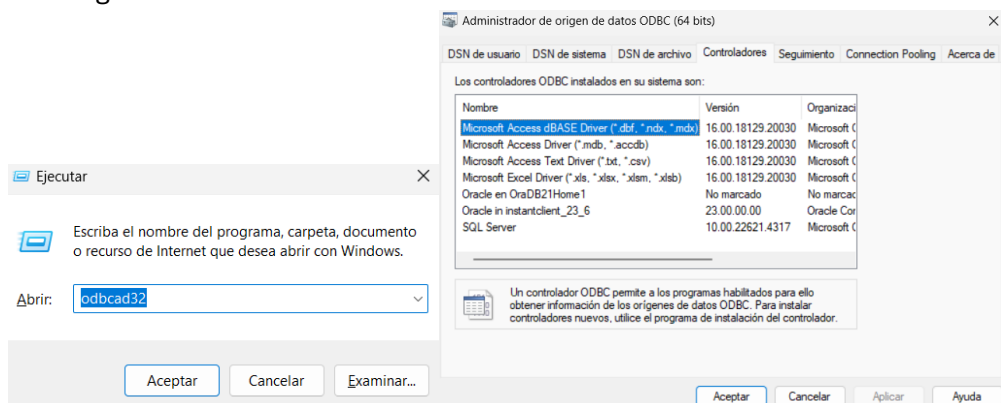
5. Descomprimos el archivo descargado en el paso 3 dentro del directorio creado en el paso anterior, en nuestro caso instantclient_23_6, el cual se encuentra dentro de :C:\oracle. Si nos indica que hay archivos con el mismo nombre los omitimos.

	instantclient-odbc-windows.x64-23.6.0.24...	09/11/2024 13:42	Carpeta comprimi...	1,035 KB
--	---	------------------	---------------------	----------

6. Ahora tendremos que modificar ciertas variables de entorno. En primer lugar, crearemos una nueva variable de usuario llamada ORACLE_HOME con la dirección a la carpeta donde tengamos hecha la instalación (en nuestro caso sería C:\oracle\instantclient_23_6) y, en segundo lugar, modificaremos la variable de del sistema Path añadiendo una nueva con la misma dirección.



7. A continuación, accedemos nuevamente al directorio donde tengamos la versión de Oracle Client (en nuestro caso C:\oracle\instantclient_23_6) y ejecutamos obdc_install.exe.
8. Por último, reiniciamos el ordenador y ya estaría instalado. Podemos comprobar que se ha realizado todo correctamente a través de Windows + R escribiendo odbcad32, accediendo a los controladores y viendo si está el que hemos descargado.



Código

Para la elaboración del código decidimos separar la implementación en dos partes, la clase encargada de gestionar las interacciones y operaciones con la base de datos y la propia aplicación, de manera que nuestro seminario se compone de dos ficheros uno correspondiente a la clase GestorBD y otro a la propia aplicación incluida la interfaz gráfica.

GestorBD: Se compone por todos los atributos necesarios para la conexión a una base de datos: driver, host, server_name, port, PWD(contraseña) y UID(usuario), además de un atributo en cargado de guardar la conexión y otro el puntero. Asimismo, incluye el constructor de todos los parámetros necesarios para la conexión y todos los métodos pertinentes para las operaciones que utiliza la aplicación: conexión, transacciones, creación y borrado de tablas, getters, modificación de filas, inserción filas en las diferentes tablas y eliminación de filas.

Cabe destacar que las únicas operaciones que incluyen un commit son la creación y borrado de tablas todas las demás necesitarían hacer uso del método confirmar para hacerlas permanentes.

Conexión:

connect(): Utiliza los parámetros pasados por el constructor para iniciar la conexión con la base de datos.

close(): Cierra la conexión con la base de datos

Transacciones y cambios:

Confirmar(): Ejecuta un commit

SavePoint(nombre): Crea un savepoint con el nombre especificado

VolverCheckpoint(nombre): Te devuelve al savepoint que con el nombre que se especifique.

CancelarTransacción(): Termina la transacción ejecutando un ROLLBACK que nos devuelve al último commit.

Creación y borrado de tablas:

CrearTablas(): Crea las tablas STOCK, PEDIDO y DETALLES_PEDIDO y rellena STOCK con 10 tulpas.

BorrarTablas(): Borra las tablas

Getters:

ObtenerTabla(nombre): Devuelve una una matriz con todas las filas de una tabla especificada.

NombresTabla(nombre): Devuelve todos los nombres de las columnas de una tabla especificada.

NumElementosTabla(nombre): Devuelve el número de filas de una tabla especificada.

ObtenerUnProducto(Cproducto): Devuelve la fila correspondiente a un producto en la tabla STOCK.

ObtenerUnPedido(Cpedido): Devuelve la fila correspondiente a un pedido de la tabla PEDIDO.

ObtenerUnDetalle_Pedido(Cpedido, Cproducto): Devuelve la fila correspondiente a un pedido de un producto.

Modificación de filas:

Modificar_Stock(Cproducto, NuevaCantidad): Cambia el atributo cantidad del producto especificado por la nueva cantidad especificada

Inserción de filas:

Insertar_Stock(Cpedido, Cproducto): Insertar una fila en la tabla STOCK con los atributos especificados.

Insertar_PEDIDO(Cpedido, Ccliente, Fecha_Pedio): Insertar una fila en la tabla PEDIDO con los atributos especificados.

Insertar_Detalle_Pedido(Cpedido, Cproducto, Cantidad): Insertar una fila en la tabla DETALLE_PEDIDO con los atributos especificados.

Eliminación de filas:

Eliminar_Stock(Cproducto): Borra la fila de STOCK correspondiente al Cproducto especificado.

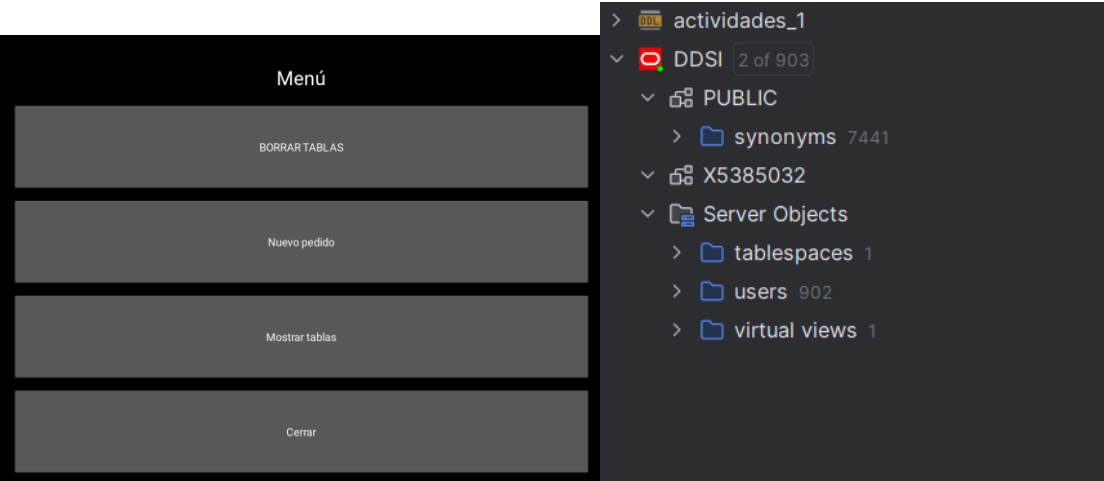
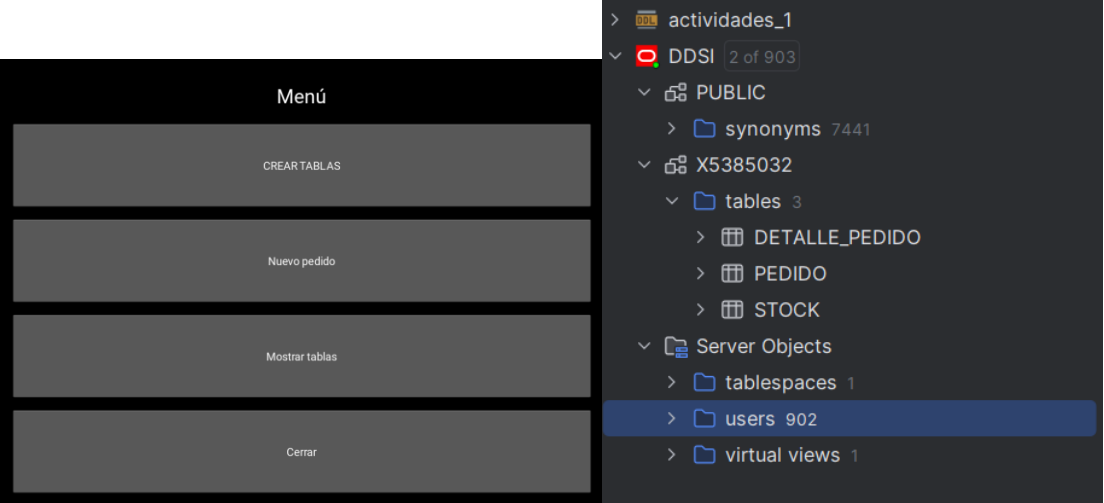
Eliminar_Pedido(Cpedido): Borra la fila de PEDIDO correspondiente al Cpedido especificado.

Eliminar_DetallePedido(Cpedido): Borra la fila de DETALLE_PEDIDO correspondiente a la combinación Cpedido y Cproducto especificada.

Aplicación e interfaz: En el segundo fichero vendrían definidas todos los detalles de la interfaz y la lógica para ir llamando a los diferentes métodos especificados en GestorBD, dado que en su mayoría es interfaz indagaremos en el apartado de funcionalidades.

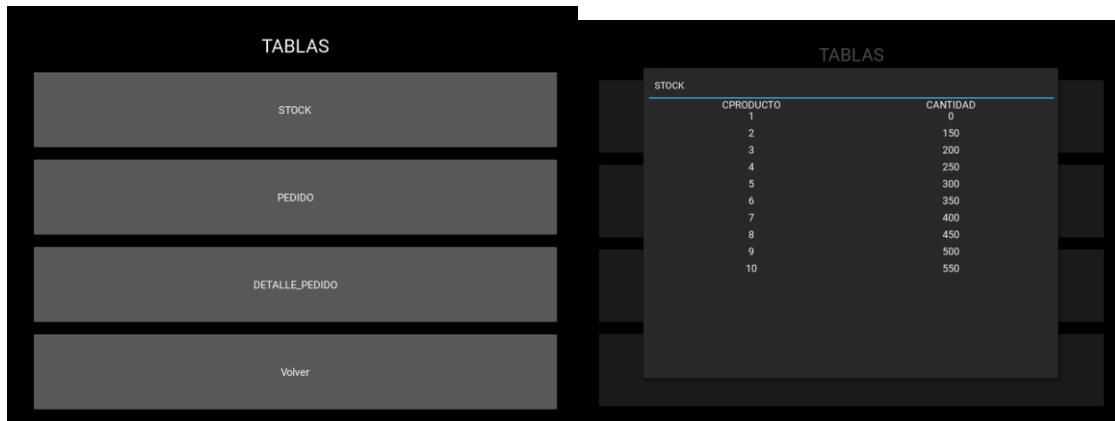
Funcionalidades

- **Borrado y Eliminación de tablas:** Ambos se tratan del mismo botón que va alternando entre crear y borrar las tablas para ello utilizamos los métodos CrearTablas() y BorrarTablas de GestorBD.



STOCK	
CPRODUCTO	CANTIDAD
1	0
2	150
3	200
4	250
5	300
6	350
7	400
8	450
9	500
10	550

- **Visualización de tablas:** Desde el menú se puede acceder a otra ventana para visualizar todas las tablas para lo que utilizamos el método de ObtenerTabla(nombre) de GestorBD.



- **Creación de un nuevo pedido:**
 1. **Ingresar datos a insertar en PEDIDO:** Al iniciar un nuevo pedido es obligatorio ingresar el Cpedido y el Ccliente, aunque, se almacena también la fecha pero esta es obtenida internamente a través de la biblioteca datetime y el comando datetime.now() el cual ya incluye la estructura del tipo date de SQL.

Nuevo pedido

Confirmar

En el caso de ser ingresado correctamente se inicia la transacción insertando una fila en la tabla PEDIDO con la información especificada y la fecha y hora. A su vez, se crea un savepoint “inicio” el cual será utilizado posteriormente para modificar los datos a insertar en la tabla DETALLE_PEDIDO y editar la cantidad de la tabla STOCK. Cabe destacar que la inserción no se ve reflejada todavía en la base de datos ya que no hemos hecho commit.

2. **Ingresar datos a insertar en DETALLE_PEDIDO y modificar STOCK:** Una vez ingresados los datos a ingresar en la tabla PEDIDO, pasamos a otra ventana en la que deberemos especificar que producto deseamos y que cantidad para posteriormente darle a enviar, botón que se encarga de insertar en la tabla DETALLE_PEDIDO el Cpedido, Cproducto y Cantidad de la fila a insertar, asimismo modifica la tabla STOCK cambiando la cantidad del producto

seleccionado por la cantidad menos la cantidad seleccionada. Cabe destacar que comprueba si la cantidad deseada es válida, es decir, si hay stock suficiente y se lo notifica al usuario.

The image displays two screenshots of a web form titled "PEDIDO".

The first screenshot shows the form with a quantity of 5 and a price of 27. Below the input fields are two buttons: "CONFIRMAR PEDIDO" and "CANCELAR".

The second screenshot shows the same form but with a quantity of 100000. An "Error" dialog box is displayed in the center, indicating "No hay stock" (No stock). The dialog has a "Cerrar" (Close) button. The background form is dimmed.

3. Modificar datos que se han insertado en DETALLE_PEDIDO: Si queremos modificar los datos que acabamos de introducir bastaría con seleccionar otro producto o cambiar la cantidad y volver a pulsar enviar, en ambos casos se llamaría al método `SavePoint()` con el parámetro "inicio" deshaciendo la inserción previa al regresar al punto nada más terminar la inserción en la tabla PEDIDO ejecutándose un rollback al savepoint creado en la funcionalidad 1 de este apartado.

The image shows a screenshot of the "PEDIDO" form with a quantity of 8 and a price of 12. Below the input fields are two buttons: "CONFIRMAR PEDIDO" and "CANCELAR".

4. Confirmar pedido: Una vez confirmados los detalles del pedido deseados se puede confirmar el pedido, lo cual ejecutaría el método Confirmar() de GestorBD (ejecuta un commit) finalizando exitosamente la transacción y reflejándose los cambios en la base de datos. Nos regresaría al menú inicial.

Menú		
CREAR TABLAS		
Nuevo pedido		
Mostrar tablas		
Cerrar		

STOCK		
CPRODUCTO	CANTIDAD	
1	0	
2	150	
3	200	
4	250	
5	300	
6	350	
7	391	
8	380	
9	500	
10	550	

PEDIDO		
CPEDIDO	CCLIENTE	FECHA_PEDIDO
13	12	2024-11-24 02:11:24

DETALLE_PEDIDO		
CPEDIDO	CPRODUCTO	CANTIDAD
13	8	12

5. Cancelar pedido: Se finalizaría la transacción a través del método CancelarTransacción() (ejecuta un ROLLBACK) y regresaría al menú inicial sin haberse efectuado ningún cambio.

Cerrar aplicación: Cierra la aplicación y la conexión con la base de datos a través del método cerrar() GestorBD

Reparto:

Diego Cabrera: Lógica e implementación de las transacciones.

Álvaro García: Métodos encargados de la creación y eliminación de tablas, así como los referentes a las inserciones en las tablas, getters y modificación.

Rafael Rivas: Investigación sobre la conexión con la base de datos e implementación de todo lo relacionado con la conexión y desarrollar la interfaz gráfica.